



Devoir 5 : remise le 27 avril avant minuit

Les objectifs de ce devoir sont de se familiariser avec :

- les analyses graphiques pour mettre en évidence les composantes d'une série chronologique ;
- la décomposition STL pour analyser et faire la prévision ;
- le processus ARIMA et le lissage exponentiel pour faire la prévision.

Ce devoir noté sur 10 comporte deux parties valant respectivement 2 points et 7 points, $\frac{1}{2}$ point étant attribué à la qualité de la présentation du rapport (sous forme de présentation) et $\frac{1}{2}$ point pour le code à fournir en annexe.

Dans tout le devoir, assurez-vous de n'utiliser que la librairie fpp3 qui accompagne le manuel de référence pour la partie du cours sur les séries chronologiques.

Partie A : Analyses préliminaires (2 points)

Dans cette partie, l'évaluation porte sur

- (1) l'illustration et la discussion des caractéristiques de la série chronologique à modéliser.
- (2) la justification des choix de paramétrage utilisé pour la décomposition STL ;
- (3) la mise en place d'une approche de référence simple.

Préparation : lecture des données

Vous allez travailler sur la série mensuelle des températures moyennes de l'Angleterre centrale (Central England Temperature, CET). Il s'agit de la plus longue série instrumentale de températures au monde, disponible depuis 1659 et mise à jour mensuellement par le Met Office britannique. Les données sont en degrés Celsius et représentent des températures moyennes mensuelles pour une région triangulaire de l'Angleterre délimitée par le Lancashire, Londres et Bristol.

Les données sont librement disponibles à l'adresse <https://www.metoffice.gov.uk/hadobs/hadcet/> et vous sont également fournies avec le devoir en format texte. Le code ci-dessous montre comment les lire dans R, créer une série d'entraînement couvrant la période 1659-1996 et une période de test couvrant les années 1997-2026 (environ 30 ans, un cycle climatique) :

```
library(fpp3)
library(readr)
library(lubridate)

## Lecture et mise en forme des données CET mensuelles
cet_raw <- read.table("~/Data/meantemp_monthly_totals.txt", header = TRUE, skip = 4)

ts_cet <- cet_raw %>%
  select(-Annual) %>%
  pivot_longer(Jan:Dec, names_to = "Month", values_to = "Tmean") %>%
  mutate(date = yearmonth(paste(Year, Month), "%Y %b")) %>%
  filter(Tmean != -99.9) %>%
  as_tsibble(index = date)

## Série d'entraînement: 1659-1996
cet.train <- ts_cet |> filter(Year <= 1996) |> as_tsibble(index = date)

## Série de test: 1997-2026
cet.test <- ts_cet |> filter(Year >= 1997) |> as_tsibble(index = date)
```

Évaluation – élément (1) (1/2 point)

Vous allez réaliser des analyses graphiques pour mettre en évidence les composantes de la série chronologique réservée pour l'entraînement. Pour cela, adaptez le code ci-dessous :

```
## Graphique temporel
autoplot(cet.train, Tmean, linewidth = 0.8) + labs(y = "Température (°C)", x = "Mois", title = "CET mensuelle (1900-2020)") + theme(text = element_text(size = 20))

## Graphiques saisonniers
cet.train |> gg_season(Tmean, labels = 'both', linewidth = 0.8) +
labs(y = "Température (°C)", x = "Mois") + theme(text = element_text(size = 20))

cet.train |> gg_subseries(Tmean) + theme(text = element_text(size=20))

## graphique de lag: tester plusieurs lags en changeant la valeur de
# l'argument "lags"
cet.train |> gg_lag(Tmean, geom = 'point', lags = c(1, 6, 12)) +
theme(text = element_text(size = 20), legend.title = element_blank())

## Graphique d'auto-corrélation (jusqu'à 36 lags pour voir 3 cycles)
cet.train |> ACF(Tmean, lag_max = 36) |> autoplot() + theme(text = element_text(size = 20))
```

Pour ces analyses graphiques, l'important est de commenter les caractéristiques (tendance à long terme, saisonnalité, bruit) que vous identifiez dans chacun des types de graphiques, en gardant en tête que ces caractéristiques vont servir à la modélisation et à la prévision.

Évaluation – élément (2) (1 point)

Vous allez ensuite effectuer une décomposition STL et évaluer si la composante résiduelle peut être considérée ou non comme un bruit blanc en vous basant sur l'auto-corrélation empirique :

```
cet.train |> model(STL(Tmean ~ trend(window = 24) + season(window = 'periodic')) |> components() -> cet.stl)

## Graphique standard de la décomposition
autoplot(cet.stl)

## Graphique de l'ACF de la composante résiduelle
cet.stl |> ACF(remainder, lag_max = 24) |> autoplot()
```

La décomposition STL ci-dessus suppose que la composante saisonnière est périodique. Utilisez une composante saisonnière qui s'adapte en changeant la valeur de l'argument window de la fonction season pour des valeurs numériques associées au niveau de lissage. Vérifiez si l'auto-corrélation de la composante résiduelle s'apparente davantage à celle d'un bruit blanc. Testez aussi quelques valeurs des deux paramètres de lissage window dans les fonctions trend et season. Incluez dans votre rapport les essais les plus informatifs, en particulier pour évaluer si le niveau de lissage retenu est acceptable et si la fenêtre saisonnière doit être périodique.

Évaluation – élément (3) (1/2 point)

Vous allez mettre en place une approche de référence simple : la méthode saisonnière naïve. Utilisez le code ci-dessous :

```
## Méthode naïve saisonnière stocké dans l'objet cet.benchmark
cet.benchmark <- cet.train |> model('Naive saison.' = SNAIVE(Tmean))

## Prévision sur 12 mois avec le modèle naïf
cet.benchmark.fc <- cet.benchmark |> forecast(h = 12)

## Graphique avec intervalle de prévision 95%
cet.benchmark.fc |> autoplot(cet.train |> filter(Year >= 1994),
level= 95, linewidth = 1.2 ) + autolayer(cet.test |> filter(Year <= 1997), Tmean, linewidth = 1.2, linetype = 2 ) + labs(y = "degrés C", x = "mois") + theme(text = element_text(size = 20))
```

Ce modèle vous servira de référence en termes de comparaison avec les approches développées dans la partie B.

Partie B : modélisation, prévision et comparaison (7 points)

Dans cette partie, l'évaluation porte sur

- (1) La proposition et le développement de trois modèles pertinents pour la modélisation de la série de température moyenne mensuelle;
- (2) les analyses des résidus et la discussion de leurs propriétés;
- (3) la mise en place d'une méthode de validation séquentielle adaptée et
- (4) l'évaluation et la comparaison de plusieurs méthodes de prévision.

Évaluation – élément (1) (3 points)

Sur la base des techniques vues en cours uniquement, proposez trois modèles de séries chronologiques qui semblent pertinents pour modéliser la série d'intérêt. Justifiez vos choix de modèles. Proposez une implémentation à l'aide de la librairie fpp3.

Évaluation – élément (2) (1.5 points)

Pour chacun des trois modèles que vous avez proposés, vous devez d'abord déterminer si les résidus ont les propriétés attendues. Inspirez-vous de la démarche ci-dessous appliquée au modèle de référence de la partie A :

```
## Analyse graphique des résidus
cet.benchmark |> gg_tsresiduals()

## Test portmanteau de Ljung-Box avec 10 lags
cet.benchmark |> augment() |> features(.innov, ljung_box, lag = 10)
```

À partir de ces analyses (graphique et test statistique de portmanteau), vous devez établir quels modèles, parmi les trois proposés, fournissent des résidus ayant les propriétés adéquates pour la modélisation. Ce seront les modèles à retenir pour la prochaine étape.

Évaluation – élément (3) (1.5 points)

Proposez et justifiez une méthode de validation séquentielle parmi celles présentées en cours. Cette méthode devra permettre de couvrir l'ensemble de test en vue d'évaluer et de comparer les performances des modèles de prévision. Proposez une implémentation à l'aide de la librairie fpp3.

Évaluation – élément (4) (1 point)

Comparez les performances de prévision, sur l'ensemble de test à l'aide de la méthode de validation séquentielle proposée, des modèles retenus parmi les trois proposés, ainsi que du modèle de référence de la partie A.

Parmi les informations à prendre en compte, inspirez-vous du code ci-dessous :

```
## Graphique avec intervalle de prévision 95%
cet.benchmark.fc |> autoplot( cet.train |> filter(Year >= 1994),
level= 95, linewidth = 1.2 ) + autolayer( cet.test |> filter(Year <=
1997), Tmean, linewidth = 1.2, linetype = 2 ) + labs(y = "degrés C", x
= "mois") + theme(text = element_text(size = 20))

## Affichage des paramètres
report(cet.benchmark)

## Mesures de performance sur la série de test
cet.benchmark |> forecast(h = 48) |> accuracy(cet.test)
```

Dans votre rapport, rassemblez les mesures de performances pertinentes pour les différents modèles considérés dans un tableau et présentez des graphiques mettant en évidence les propriétés des modèles. N'oubliez pas d'inclure la modèle de référence de la partie A, c'est-à-dire la méthode naïve saisonnière. Déterminez le modèle qui vous paraît le plus approprié pour la prévision et justifiez.